

The methodology in the paper (e.g., Steps 3–4 for address matching and derivatives) relies heavily on linking properties via shared or similar owner addresses, which often vary due to data entry errors, abbreviations, formatting differences, or minor inconsistencies (e.g., "5001 Plaza on the Lake Dr" vs. "5001 Plaza On Lake Drive Suite 200"). Since you're dealing with large volumes of such "close but not exact" addresses and want a free, self-hosted system centered on PostGIS (PostgreSQL's spatial extension), I'll outline viable options below. These focus on address normalization (standardizing format/components) and fuzzy matching (identifying similarities).

The goal is to:

- **Normalize:** Parse and standardize addresses (e.g., expand "St" to "Street", handle suite variations).
- **Match:** Use string similarity, geospatial proximity (if geocoded), or clustering to group close variants.
- **Keep it free:** No paid APIs (e.g., no Google Maps or SmartyStreets). Rely on open-source libraries, data sources, and self-hosted tools.

Core Setup: PostGIS + PostgreSQL

PostGIS is an excellent backbone—it's free, open-source, and handles spatial data natively. You'll store addresses in a PostgreSQL table, normalize them, optionally geocode for proximity matching, and query for matches using built-in fuzzy logic.

1. Installation:

- Install PostgreSQL (free from postgresql.org) and enable PostGIS extension (via package manager: e.g., `sudo apt install postgis` on Ubuntu).
- Create a database: `createdb mydb; psql mydb -c "CREATE EXTENSION postgis; CREATE EXTENSION fuzzystmatch; CREATE EXTENSION pg_trgm;"`
 - `fuzzystmatch`: For Levenshtein distance (edit distance) matching.
 - `pg_trgm`: For trigram similarity (fast, effective for fuzzy string matching).
- Load your data: Use `psql` or pgAdmin to import CSV/JSON with addresses into a table like:

```
SQL

CREATE TABLE addresses (
    id SERIAL PRIMARY KEY,
    raw_address TEXT,
    normalized_address TEXT,
    geom GEOMETRY(Point, 4326) -- For lat/long if geocoding
);
```

2. Address Normalization Options

Normalization is key to handling variations. Process addresses in batch via scripts, then store normalized versions in PostGIS for querying.

- **libpostal (Best Free Option for Global/US Addresses):**
 - This is a C library from OpenCage/OpenVenue, trained on millions of addresses. It parses into components (house number, road, city, state, postcode, etc.) and normalizes (e.g., "Ave" to "Avenue", handles PO Boxes).
 - Free and open-source (MIT license). No API keys/limits.

- **How to Use:**

- Install: `sudo apt install libpostal-dev` (or build from source: github.com/openvenues/libpostal).
- Python wrapper: `pip install postal` (`pypostal`).
- Script example to normalize a list of addresses:

Python

```
from postal.parser import parse_address
from postal.expand import expand_address

addresses = ["5001 Plaza on the Lake Dr Suite 200 Austin TX", "5001 plaza on lake drive #200 austin texas"] # Your data

for addr in addresses:
    expanded = expand_address(addr) # Returns normalized variants
    parsed = parse_address(expanded[0]) # Pick best, parse into dict
    normalized = f"{parsed.get('house_number', '')} {parsed.get('road', '')} {parsed.get('unit', '')} {parsed.get('city', '')} {parsed.get('state', '')} {parsed.get('postcode', '')}"
    print(normalized) # Output: "5001 Plaza On The Lake Drive Suite 200 Austin Texas" (standardized)

# Batch insert normalized into PostGIS via psycopg2
import psycopg2
conn = psycopg2.connect("dbname=mydb user=youruser")
cur = conn.cursor()
cur.execute("INSERT INTO addresses (raw_address, normalized_address) VALUES (%s, %s)", (addr, normalized))
conn.commit()
```

- Pros: Handles international addresses, fuzzy inputs, very accurate for US (trained on OpenStreetMap + other data). Fast for batches.

- Cons: Setup requires compiling if not using packages; memory-intensive for millions of records (but fine for PostGIS integration).

- **usaddress (US-Specific Parser):**

- Free Python library (github.com/datamade/usaddress) from DataMade. Parses US addresses into components and tags them (e.g., distinguishes "Suite" from "Apt").
- Install: `pip install usaddress`.
- Example:

```
Python

import usaddress
addr = "5001 Plaza on the Lake Dr, Ste 200, Austin, TX 78746"
parsed, addr_type = usaddress.tag(addr)
normalized = ' '.join([str(v) for v in parsed.values() if v])
# Customize as needed
```

- Combine with `fuzzywuzzy` (`pip install fuzzywuzzy`) for post-parsing similarity scoring.
- Pros: Lightweight, no dependencies. Great for US-only data like your Atlanta-focused example.
- Cons: Less robust than `libpostal` for non-standard formats; no built-in expansion/normalization.

- **Pure SQL Normalization in PostGIS:**

- If you prefer no external libs, use `regex/replace` functions in SQL for common fixes (e.g., replace "Dr" with "Drive").

```
SQL

UPDATE addresses SET normalized_address =
  regexp_replace(raw_address, '\bDr\b', 'Drive', 'gi');
```

- Limited for complex cases; better as a supplement.

3. Fuzzy Matching Options

Fuzzy Matching Options

Once normalized, query PostGIS to find matches. Focus on string similarity first, then geospatial if needed.

- **String-Based Matching (Built into PostgreSQL):**

- Use `pg_trgm` for similarity scores (0–1 scale):

```
SQL

SELECT a1.raw_address, a2.raw_address,
similarity(a1.normalized_address, a2.normalized_address) AS
sim_score
FROM addresses a1
JOIN addresses a2 ON a1.id < a2.id -- Avoid self-joins
WHERE similarity(a1.normalized_address,
a2.normalized_address) > 0.8; -- Threshold for "close"
```

- Or Levenshtein distance:

```
SQL

SELECT *, levenshtein(normalized_address, '5001 Plaza On The
Lake Drive Suite 200 Austin Texas') AS distance
FROM addresses WHERE levenshtein(normalized_address, '5001
Plaza On The Lake Drive Suite 200 Austin Texas') < 5;
```

- Cluster matches: Use recursive queries or Python (e.g., with `networkx`) to group into networks like in the paper's Figure 1.
- Pros: Fast, scalable for large datasets (index with `CREATE INDEX ON addresses USING GIST (normalized_address gist_trgm_ops);`).
- Threshold tuning: Start with 0.8–0.9 similarity; manually review edge cases.

- **Geospatial Matching (for Proximity-Based Confirmation):**

- Geocode normalized addresses to lat/long, then match if strings are similar *and* points are close (e.g., same building).

- **Free Geocoder: PostGIS TIGER Geocoder** (US-only, perfect for your case):

- Download free US Census TIGER data (census.gov/geo/tiger), load into PostGIS (scripts at github.com/geocompx/postgis-tiger-geocoder).
- Install: Enable extension `CREATE EXTENSION tiger;`, then geocode:

```
SQL

SELECT pprint_addy(normalize_address('5001 Plaza on the
Lake Suite 200 Austin TX')) AS norm_addy,
       geocode('5001 Plaza on the Lake Suite 200 Austin
TX') AS geom;
UPDATE addresses SET geom = geocode(normalized_address);
```

- Match by distance:

```
SQL

SELECT a1.raw_address, a2.raw_address,
       ST_Distance(a1.geom, a2.geom) AS dist_meters
FROM addresses a1 JOIN addresses a2 ON ST_DWithin(a1.geom,
a2.geom, 50); -- Within 50m
```

- Alternative: Self-hosted Nominatim (OSM-based, global). Download OSM data (planet.openstreetmap.org), import to PostGIS (nominatim.openstreetmap.org/docs), query via API or SQL. Setup time: 1–2 days for US extract.
- Pros: Confirms "close" addresses are physically the same (e.g., suite variations in same building). Handles zip/state mismatches.
- Cons: TIGER/Nominatim may fail on 5–10% of addresses; fallback to string-only.

4. Automation and Scaling

Automation and Scaling

- **Batch Processing:** Use Python (with psycopg2) to normalize/geocode in loops, insert to PostGIS.
- **Clustering Networks:** For paper-like ownership networks, use Python's networkx (pip install networkx) to build graphs from matches:

```
Python

import networkx as nx
G = nx.Graph()
# Add edges between matched addresses from SQL query
# Then find connected components for groups
```

- **Handling Volume:** PostGIS scales to millions of records. Use partitioning for huge datasets.
- **Testing:** Start with a subset (e.g., Atlanta addresses from the paper). Validate against known matches like "5001 Plaza on the Lake" vs. derivatives.

5. Potential Challenges and Tips

- **Accuracy:** Combine methods (e.g., libpostal + trigram + geocoding) for 90–95% automation; manual review for high-value matches.
- **US Focus:** All tools above excel for US addresses; if international, libpostal/Nominatim handle it.
- **Resources:** Docs at postgis.net, libpostal readme. Communities: Stack Overflow, GIS Stack Exchange.
- **Alternatives if PostGIS Feels Heavy:** DuckDB (free, in-memory SQL) with spatial extension + libpostal for lighter setups.
- **Legal/Ethical:** Ensure your data source complies with usage terms (e.g., TIGER/OSM are public domain).

This setup should get you close to automating the address matching in the methodology. If you provide sample addresses or more details (e.g., data volume), I can refine scripts.